Project Report On

Containerized Marketing Service with Docker

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Bhavya Rathore	EN22CS301269
2	Bhoomika Sanodiya	EN22CS301271
3	Bhumika Chouhan	EN22CS301275
4	Devi Sanjaykumar Modi	EN23CS3T1008
5	Dollyna Sahu	EN22CS301354

Group Name: Group G2D3

Project Number: DO-26

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-2026

1. Problem Statement & Objectives.

1.1 Problem Statement

In modern software development, deploying applications efficiently and reliably is a major challenge due to differences in environments, manual deployment processes, and lack of scalability. Traditional deployment methods often lead to inconsistencies, increased errors, and difficulty in maintaining applications across different platforms.

Additionally, AI-based applications require a robust and scalable infrastructure to handle real-time processing and ensure high availability. Without proper DevOps practices, managing application deployment, updates, and scaling becomes complex and inefficient.

Therefore, there is a need for a solution that:

- Ensures consistent deployment across environments
- Automates and simplifies the deployment process
- Provides scalability and reliability using cloud infrastructure
- Uses containerization and orchestration for efficient resource management
- Enables seamless integration of AI services in a production environment

To address these challenges, there is a need for a system that leverages DevOps practices to ensure:

- Automated and consistent deployment
- Containerized application environments
- Scalable and reliable cloud infrastructure
- Efficient integration of AI services
- Simplified management and orchestration of applications

This project aims to solve these problems by implementing a DevOps-driven deployment pipeline using Docker, DockerHub, Kubernetes, and AWS EC2.

1.1 Project Objectives

The primary objective of this project is to implement a complete DevOps-based deployment pipeline for an AI-powered Marketing Agent application using modern tools and cloud technologies.

Specific Objectives:

- To develop an AI-powered application using Python, Streamlit, and Groq API
- To containerize the application using Docker for environment consistency
- To store and manage Docker images using DockerHub
- To deploy the application on cloud infrastructure using AWS EC2
- To implement Kubernetes (Minikube) for container orchestration
- To ensure scalability, portability, and reliability of the application
- To follow DevOps best practices for deployment and infrastructure management

1.3 Project Scope

The scope of this project defines the boundaries within which the system is designed, developed, and deployed using DevOps practices. It focuses on implementing a streamlined and practical DevOps workflow for a containerized AI application, covering development, deployment, and basic orchestration in a controlled environment.

The project emphasizes demonstrating real-world DevOps concepts such as containerization, cloud deployment, and orchestration, while keeping the system simple, lightweight, and easy to understand for academic and practical purposes.

Implementation of a containerized deployment model : The project includes packaging the application using Docker to ensure consistent behavior across development and production environments.

Cloud-based hosting using virtual infrastructure : Deployment is performed on AWS EC2 to simulate a real-world production environment with public accessibility.

Basic orchestration using Kubernetes (Minikube): The system demonstrates container orchestration concepts such as deployment and service management in a local Kubernetes cluster.

Integration of AI services within a DevOps workflow : The application integrates external AI APIs (Groq) and manages them within a containerized and cloud-based system.

Use of version control and configuration management: The project includes maintaining code, Docker configurations, and Kubernetes YAML files in a version-controlled repository for reproducibility.

2. Proposed Solution

2.1 Key Features

The key features of this project covers the implementation of DevOps principles for building, deploying, and managing a cloud-based AI application.

Application Layer:

- Development of AI Marketing Agent using Python and Streamlit
- Integration with Groq API for dynamic content generation
- User interaction through web-based UI

DevOps Implementation:

Containerization:

- Creation of Dockerfile
- Building and running Docker containers
- Ensuring portability and consistency

Image Management:

- Pushing Docker images to DockerHub
- Pulling images from registry during deployment

Cloud Deployment:

- Setting up AWS EC2 instance
- Installing Docker on EC2
- Running containerized application

Orchestration:

- Writing Kubernetes configuration files:
    - deployment.yaml
    - service.yaml
- Deploying application using kubectl

Version Control:

- Using GitHub for code management
- Storing Dockerfile, YAML files, and README

- Maintaining project documentation

2.1 Overall Architecture / Workflow

The overall architecture of the AI Marketing Agent is designed using a layered and modular DevOps-based approach, where each component is responsible for a specific function in the system. The architecture integrates application development, containerization, cloud deployment, and orchestration to ensure scalability, portability, and reliability.

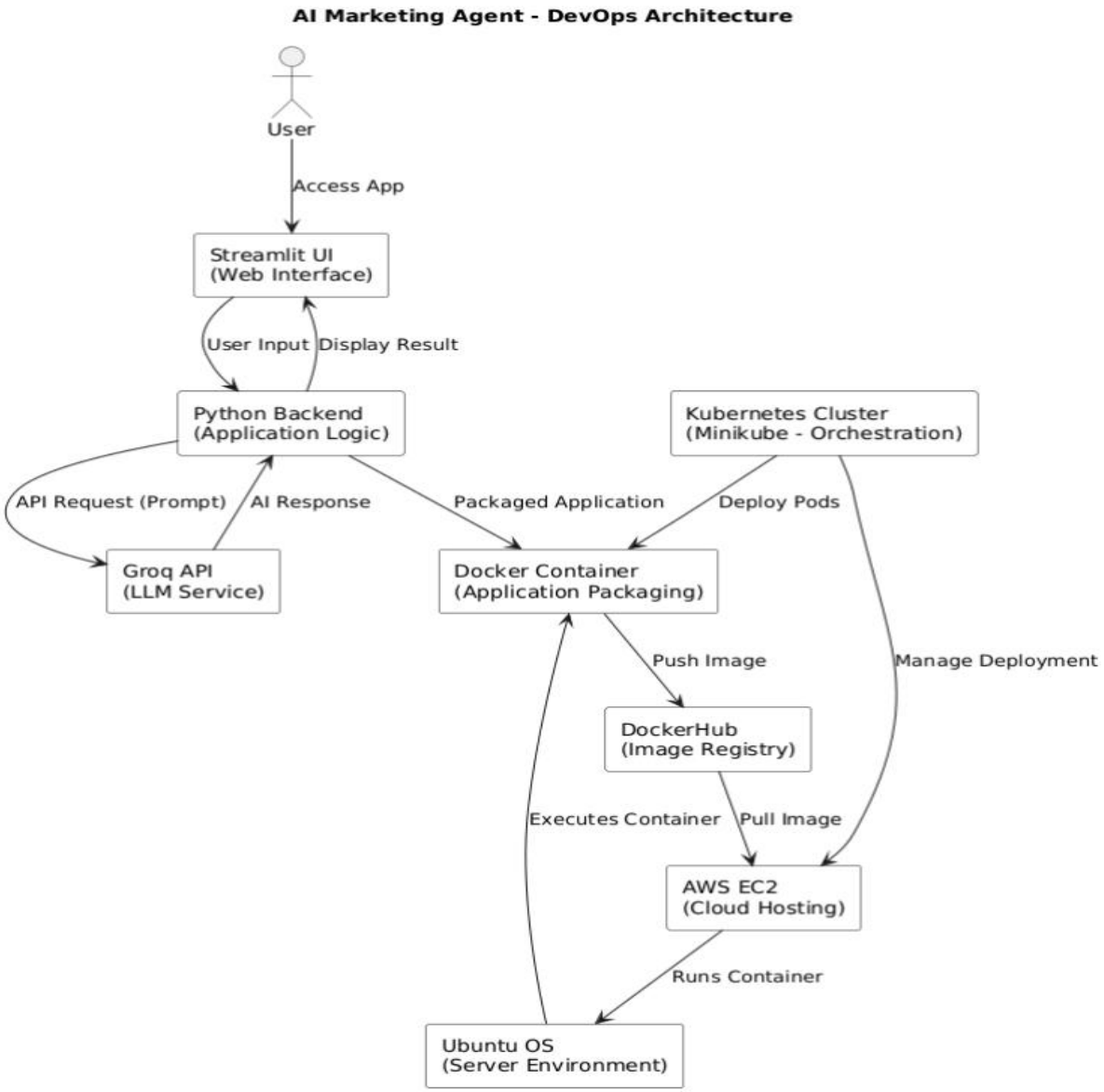

Figure 1: Architecture Diagram

Workflow

1. Application Workflow (Runtime Flow)

This workflow describes how the system processes user input and generates output in real time.

User → Streamlit UI → Python Backend → Groq API → Response → UI Display

1. **User Interaction**
   - The user accesses the application via browser
   - Enters a marketing prompt in the Streamlit UI
2. **Request Handling (UI → Backend)**
   - Streamlit forwards the input to the Python backend
3. **Processing & API Call**
   - Backend processes the input
   - Sends request to Groq API
4. **AI Response Generation**
   - Groq API generates marketing content using LLM
5. **Response Delivery**
   - Backend receives response
   - Sends output to UI
6. **Output Display**
   - Streamlit displays the generated content to the user

2. DevOps Workflow (Deployment Flow)

This workflow describes how the application is built, deployed, and managed using DevOps tools.

Code → Docker Build → DockerHub Push → AWS EC2 → Kubernetes → Running Application

1. **Code Development**
   - Application is developed using Python and Streamlit
2. **Containerization (Docker)**
   - Dockerfile is created
   - Image is built using Docker
3. **Image Storage (DockerHub)**
   - Docker image is pushed to DockerHub
   - Acts as centralized image repository
4. **Cloud Deployment (AWS EC2)**

- EC2 instance is created
- Docker is installed
- Image is pulled and container is run

5. **Orchestration (Kubernetes)**
    - Deployment and Service YAML files are applied
    - Kubernetes manages pods, scaling, and availability

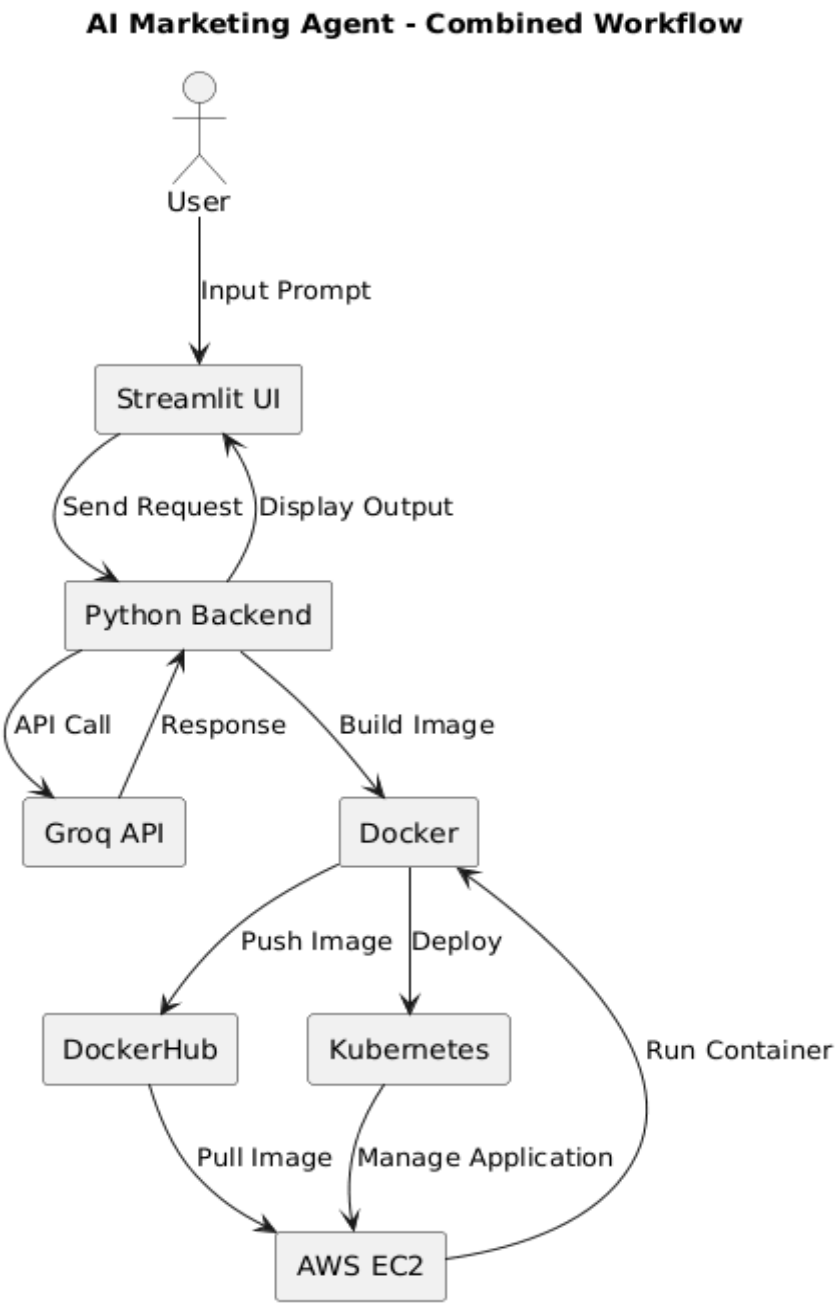

Figure 2: Workflow Diagram

2.3 Tools & Technologies

Category	Tools/Technology	Description
Core Programming	Python	Core programming language used to develop backend logic and integrate APIs
Frontend	Streamlit	Used to build an interactive web-based user interface for input/output
AI API & LLM	Groq API	Provides AI capabilities for generating marketing content in real time
Containerization	Docker	Used to package the application and its dependencies into a container
Image Registry	DockerHub	Stores and manages Docker images for easy sharing and deployment
Cloud Platform	AWS EC2	Provides virtual server infrastructure to host the application
Orchestration	Kubernetes(Minikube)	Manages container deployment, scaling, and lifecycle
Version Control	Git & Github	Used for source code management and version tracking
Configuration Management	YAML Files	Used for defining Kubernetes deployments and services
Security & Config	Environment Variables	Used to securely manage API keys and sensitive data

3. Result & Screenshot

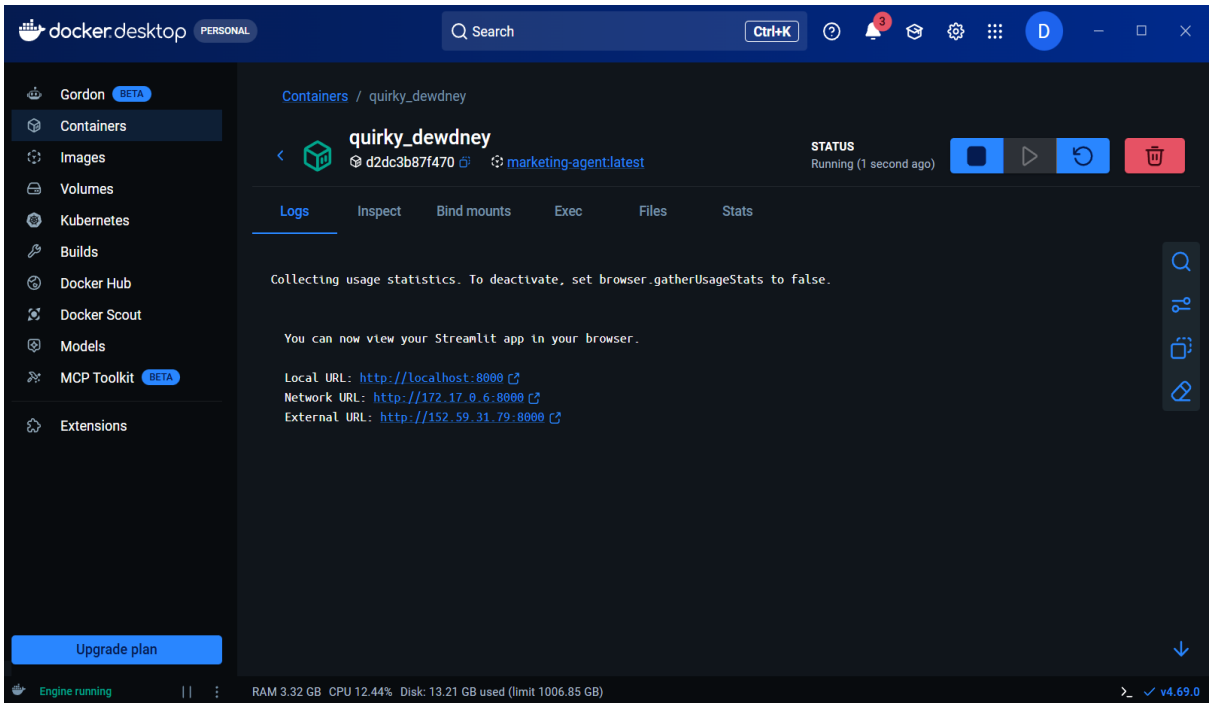

Figure 3: Screenshot of Docker Interface

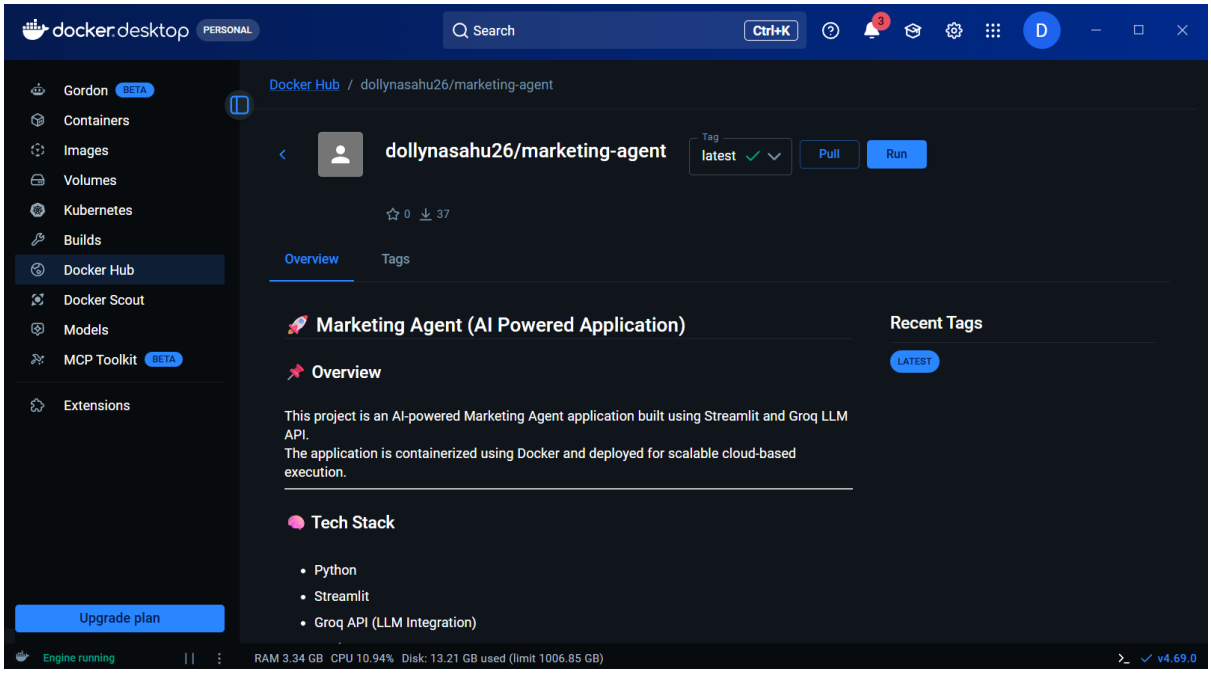

Figure 4: Screenshot of DockerHub Interface

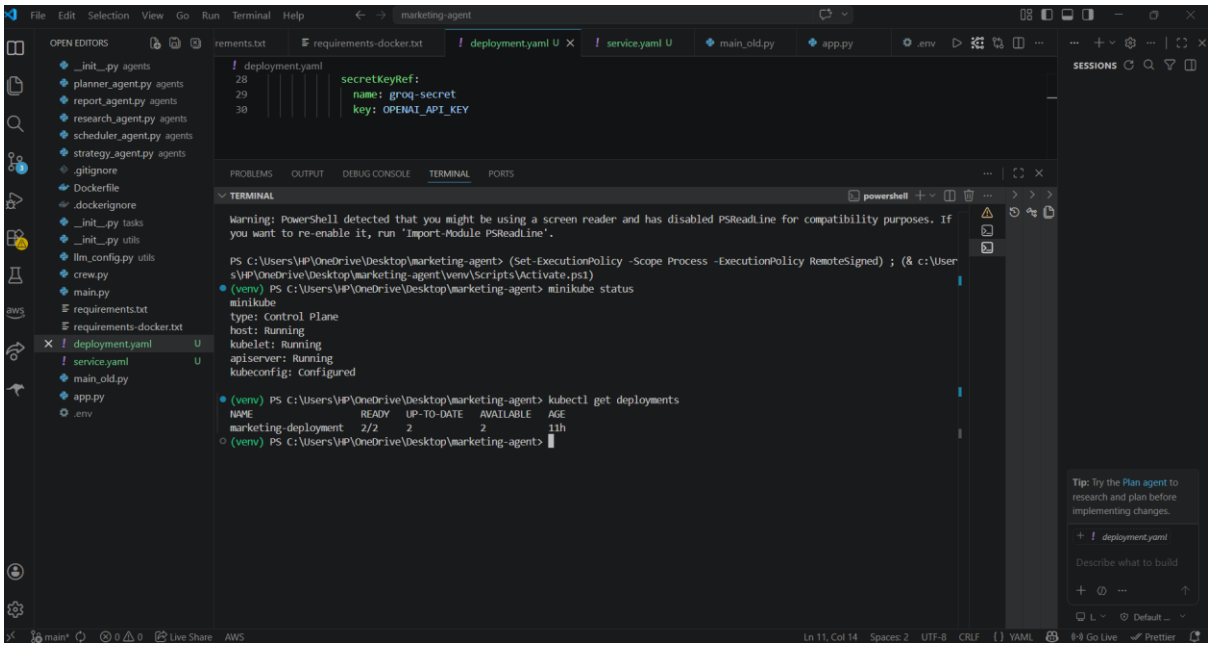

Figure 5: Screenshot of Minikube(Kubernetes Integration)

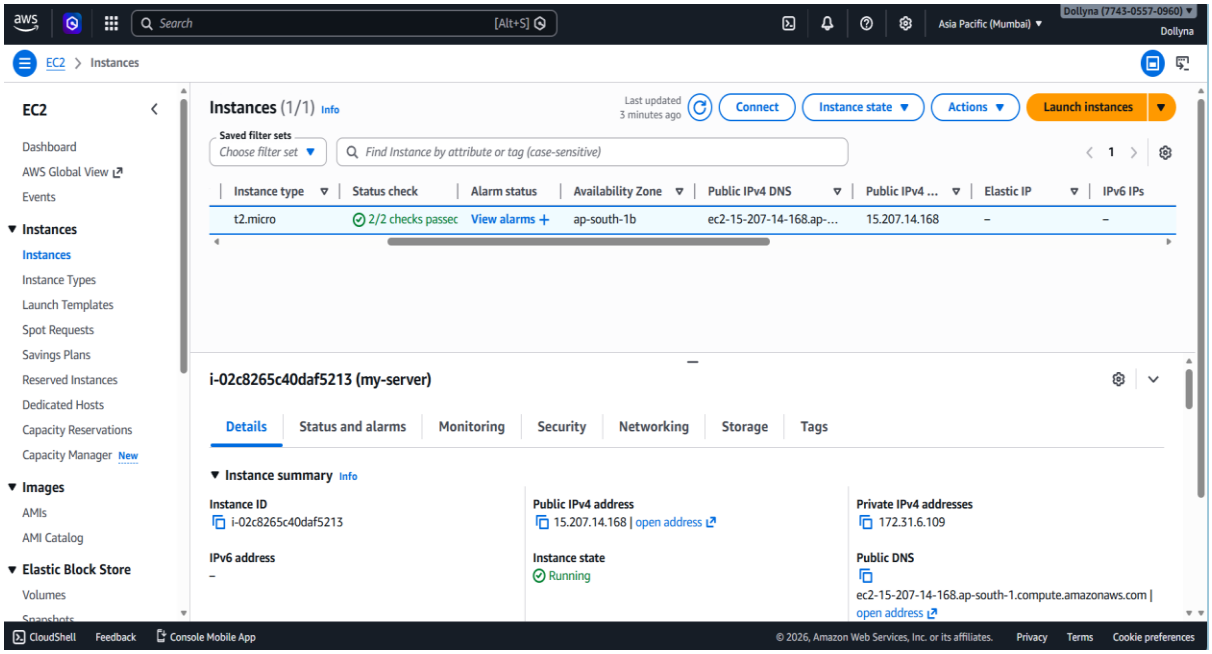

Figure 6: Screenshot of EC2 Instance Interface

Key Outcomes

The successful implementation of this project resulted in several important outcomes related to both application functionality and DevOps practices.

End-to-End DevOps Workflow Implementation : The project demonstrates a complete DevOps lifecycle, including development, containerization, image management, cloud deployment, and orchestration.

Successful Containerization using Docker : The application was packaged into a Docker container, ensuring consistent execution across different environments without dependency issues.

Cloud Deployment on AWS EC2 : The system was successfully deployed on a cloud-based virtual server, making the application accessible over the internet with continuous availability.

Integration of AI Capabilities : The application effectively integrates Groq API to generate real-time marketing content, showcasing the combination of AI with DevOps practices.

Basic Kubernetes Orchestration : Deployment and service configurations were implemented using Kubernetes (Minikube), enabling container management and demonstrating scalability concepts.

Improved Portability and Scalability : The use of containerization and cloud infrastructure ensures that the application can be easily moved and scaled as required.

4. Conclusion

This project successfully demonstrates the integration of modern DevOps practices with an AI-based application. By leveraging tools such as Docker, DockerHub, AWS EC2, and Kubernetes, the system achieves a scalable, portable, and reliable deployment architecture.

The use of containerization ensures consistency across environments, while cloud deployment provides high availability and accessibility. Additionally, Kubernetes introduces basic orchestration capabilities, enhancing the system's ability to manage and scale containerized applications.

Overall, the project highlights the importance of DevOps in simplifying application deployment and management, while also showcasing how AI-powered systems can be efficiently deployed in real-world environments using modern tools and technologies.

5. Future Enhancements

While the current implementation provides a strong foundation, several improvements can be made to enhance the system further:

CI/CD Pipeline Integration : Implement automation using tools like Jenkins or GitHub Actions for continuous integration and deployment.

Production-Grade Kubernetes Deployment : Replace Minikube with managed Kubernetes services like AWS EKS for real-world scalability and reliability.

Monitoring and Logging : Integrate tools such as Prometheus and Grafana to monitor application performance and system health.

Database Integration : Add persistent storage (e.g., MySQL, MongoDB) to store user data and generated content.

Enhanced Security Mechanisms : Implement advanced security features such as IAM roles, secret managers, and HTTPS configuration.

User Authentication System : Add login/signup functionality for personalized user experience.

Auto-Scaling and Load Balancing : Enable dynamic scaling of application instances based on traffic.